1 Verification and Testing Plan

1.1 Security Test Suite

The security test suite for Veri-Store is organized around the main integrity-critical components of the implementation: fragment hashing, homomorphic fingerprint verification, client/server protocol validation, and adversarial behavior under byzantine conditions. The goal of the suite is not only to catch regressions, but also to provide executable evidence that the implementation continues to satisfy the security claims made in this document.

1.1.1 Verification-layer unit tests

The verification package is covered by focused unit tests for:

- `RandomOracle` behavior, including deterministic output, rejection of invalid input, and correct fragment hashing behavior;
- `FingerprintedCrossChecksum` generation, serialization, digest stability, and consistency with the underlying fingerprint function;
- `Verifier.check()` and `Verifier.batch_check()`, including valid fragments, corrupted fragments, index errors, fingerprint mismatches, byzantine substitutions, and threshold-boundary cases.

1.1.2 HTTP protocol and endpoint tests

The network-facing test suite verifies that the FastAPI layer fails closed on malformed or adversarial requests. These tests cover:

- rejection of malformed `base64`, malformed `fpcc_json`, and inconsistent metadata during `PUT`;
- correct handling of duplicate writes, invalid fragment indices, and invalid verification status persistence;
- authentication requirements, rate limiting behavior, and presence of security headers on success and error responses;
- retrieval-side client behavior when servers return corrupted fragments, malformed payloads, mismatched `fpcc` values, or other untrusted responses.

1.1.3 Integration tests

In addition to unit and endpoint tests, an integration suite wires a real `VeriStoreClient` to multiple in-process server instances and verifies end-to-end security behavior. These tests confirm that:

- honest 5-server dispersal and retrieval succeeds;

- retrieval still succeeds with up to two byzantine servers in the current 3-of-5 deployment;
- retrieval fails closed once too many servers are byzantine or unavailable;
- authentication failure prevents successful dispersal.

1.1.4 Security experiments and adversarial scripts

Some security properties are better evaluated experimentally than as ordinary unit tests. For that reason, Veri-Store also includes standalone scripts that complement the automated test suite:

- a collision-probability experiment measuring empirical fingerprint and random-oracle collision rates against their theoretical expectations;
- an empirical Theorem 3.4 experiment that searches for counterexamples in which two verified fragment sets decode to different blocks under the same `fpcc`;
- a penetration-testing harness that exercises corrupted `GET` responses, malformed inputs, replay scenarios, threshold-boundary cases, and logging behavior.

1.1.5 Threat coverage

The intended maintenance rule is that each mitigated threat in Section 11 should correspond to at least one executable check in the security suite. Some threats are covered by deterministic unit or integration tests, while others are covered by repeatable experiment scripts and documented attack matrices. Together, these tests give the project a regression baseline for both ordinary bugs and security-relevant implementation drift.

1.2 Code Review Process

In review, we check four things:

- The change does not weaken integrity checks or bypass verification paths.
- Error handling is safe and returns clear HTTP status codes.
- Input validation is still strict for API payloads.
- Tests were added or updated for the changed behavior.

1.3 Regression Testing

We keep a small set of repeatable security checks:

- Byzantine fragment detection still rejects corrupted data.

- Retrieval still succeeds with only m healthy servers.
- Edge-case payload tests still pass, including binary and large inputs.